Lesson 6 Teacher's Key

Unreal

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

An Event Dispatcher on BP_Enemy, bound once per spawned instance from the Level Blueprint's existing spawn loop, targeting a dedicated BP_GameManager instance placed in the level:

```
BP_Enemy's Event Graph (inside the successful Cast branch):
Call OnEnemyDestroyed (Broadcast) -> Destroy Actor(Cast result) -> Destroy
Actor(Self)

Level Blueprint:
Event BeginPlay -> (BP_GameManager reference, dragged in from the level)
->
  For Loop -> Spawn Actor from Class(BP_Enemy, ...) ->
    Bind Event to OnEnemyDestroyed(Target = spawned Actor, Event =
BP_GameManager ref's HandleEnemyDestroyed)

BP_GameManager:
Custom Event HandleEnemyDestroyed ->
  EnemiesRemaining = EnemiesRemaining - 1
  Branch(EnemiesRemaining <= 0) -> True: Print String("You Win!", Print to
Screen = true)
```

Level 2 - Complete Worked Solution

Each spawned enemy's dispatcher is bound exactly once, at spawn time, always to the same BP_GameManager instance, and Call OnEnemyDestroyed only ever broadcasts once per enemy, right at the moment it's actually destroyed - there's no path where EnemiesRemaining could drift or the win message could double-fire.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: victory sound

Add a Play Sound 2D node using the provided sfx_win.wav right before the Print String node, inside BP_GameManager:

```
Branch(EnemiesRemaining <= 0) -> True:
  Play Sound 2D(sfx_win.wav)
  Print String("You Win!", Print to Screen = true)
```

Visual: styled win message

Set Print String's Text Color to gold and increase Duration so it's clearly a celebratory moment, not a debug message:

```
Print String("You Win!", Print to Screen = true, Text Color = (Gold),
Duration = 5.0)
```

Structure: BP_GameManager owns the counting

EnemiesRemaining and HandleEnemyDestroyed live entirely inside BP_GameManager - the Level Blueprint only ever spawns enemies and binds their dispatchers, and BP_Enemy only ever broadcasts that it was destroyed. Neither one knows or cares how many enemies remain overall.

Complete Level 3 solution (all 2 goals combined)

HandleEnemyDestroyed with both ideas added:

```
Custom Event HandleEnemyDestroyed ->
  EnemiesRemaining = EnemiesRemaining - 1
  Branch(EnemiesRemaining <= 0) -> True:
    Play Sound 2D(sfx_win.wav)
    Print String("You Win!", Print to Screen = true, Text Color = (Gold),
Duration = 5.0)
```

Alternate Path Notes: C++

A DECLARE_DYNAMIC_MULTICAST_DELEGATE plus a BlueprintAssignable UPROPERTY on ACppEnemy is the code equivalent of a Blueprint Event Dispatcher - Broadcast() called from OnOverlap() mirrors Call OnEnemyDestroyed.

The Level 3 goals map onto C++ calls the same way: UGameplayStatics::PlaySound2D() for sound, and GEngine->AddOnScreenDebugMessage() (with a custom FColor and duration) for the visual idea - both called from the GameManager actor's bound handler, mirroring BP_GameManager's role in the Blueprint path.

- Showing both C++ and Blueprint side by side for the same behavior keeps demystifying Blueprints for students who assume "visual" means "not real code" - it's the same engine calls either way.
- This is squarely the optional advanced track - most students should stay on Blueprints; C++ is for students already comfortable with typed languages from Godot, GameMaker, or Unity's C# path.